
Integration Manual

for MPC560XB ADC Driver

Document Number: IM13ADCASR4.0 Rev0003R1.0.1
Rev 1.2





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	9
2.2	Overview.....	10
2.3	About this Manual.....	10
2.4	Acronyms and Definitions.....	10
2.5	Reference List.....	11
Chapter 3		
Building the Driver		
3.1	Build Options.....	13
3.1.1	CW Compiler/Linker/Assembler Options.....	13
3.1.2	DIAB Compiler/Linker/Assembler Options.....	15
3.1.3	GHS Compiler/Linker/Assembler Options.....	17
3.1.4	HT Compiler/Linker/Assembler Options.....	18
3.2	Files required for Compilation.....	20
3.3	Setting up the Plug-ins.....	22
Chapter 4		
Function calls to module		
4.1	Function Calls during Start-up.....	25
4.2	Function Calls during Shutdown.....	25
4.3	Function Calls during Wake-up.....	25
Chapter 5		
Module requirements		
5.1	Exclusive areas to be defined in BSW scheduler.....	27
5.2	Peripheral Hardware Requirements.....	32
5.3	ISR to configure within OS – dependencies.....	33

Section number	Title	Page
5.4	ISR Macro.....	33
5.5	Other AUTOSAR modules - dependencies.....	34
Chapter 6 Main API Requirements		
6.1	Main functions calls within BSW scheduler, at which cycle (10ms, 20ms,...?).....	35
6.2	API Requirements.....	35
6.3	Calls to Notification Functions, Callbacks, Callouts.....	35
Chapter 7 Memory Allocation		
7.1	Sections to be defined in MemMap.h.....	37
7.2	Linker command file.....	38
Chapter 8 Configuration parameters considerations		
8.1	Configuration parameters considerations.....	39
Chapter 9 Integration Steps		
Chapter 10 ISR Reference		
10.1	Software specification.....	47
10.1.1	Define Reference.....	47
10.1.2	Macros Reference.....	47
10.1.2.1	Macro ADC_CTU_START_VALUE.....	47
10.1.2.2	Macro ADC_GROUP_PRIORITY.....	47
10.1.2.3	Macro ADC_GroupInQueue.....	47
10.1.2.4	Macro ADC_GroupsInHwQueue.....	48
10.1.2.5	Macro ADC_GroupTriggerSource.....	48
10.1.2.6	Macro ADC_GroupUnit.....	48
10.1.2.7	Macro ADC_STREAMING_NUM_SAMPLES.....	48
10.1.2.8	Macro CTU_LLD_ClearTRGI.....	48
10.1.2.9	Macro CTU_LLD_Init_FALSE.....	49

Section number	Title	Page
10.1.2.10	Macro CTU_LLD_Init_TRUE.....	49
10.1.2.11	Macro CTU_LLD_ReadEvtCfgRegs.....	49
10.1.2.12	Macro CTU_LLD_ResetEvtCfgRegs.....	49
10.1.2.13	Macro CTU_LLD_ResetPrescalerBits.....	50
10.1.2.14	Macro CTU_LLD_ResetSvrRegs.....	50
10.1.2.15	Macro CTU_LLD_SetCounterStartValue.....	50
10.1.2.16	Macro CTU_LLD_SetEvtCfgReg.....	51
10.1.2.17	Macro CTU_LLD_SetPrescalerValue.....	52
10.1.3	Enum Reference.....	53
10.1.4	Function Reference.....	53
10.1.4.1	Function Adc_DMAEndGroupConvUnit0.....	53
10.1.4.2	Function Adc_DMAEndGroupConvUnit1.....	53
10.1.4.3	Function Adc_EndGroupConvUnit0.....	54
10.1.4.4	Function Adc_EndGroupConvUnit1.....	54
10.1.4.5	Function Adc_WatchDogThresholdUnit0.....	55
10.1.4.6	Function Adc_WatchDogThresholdUnit1.....	55
10.1.5	Structs Reference.....	56
10.1.6	Types Reference.....	56
10.1.7	Variables Reference.....	56



Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	08-Feb-2012	Massimo De Martino	Initial Draft for Beta 0.9.0
1.1	14-May-2013	Indranil Bhattacharya	Updated for changes in RTM 1.0.0
1.2	02-June-2014	Franck Thebault	Updated for changes in RTM 1.0.1



Chapter 2

Introduction

This integration manual describes the integration requirements for ADC Driver for MPC560XB microcontrollers.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor .

Table 2-1. MPC5607B Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100 mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
-------------------------	--

All of the above microcontroller devices are collectively named as MPC5607B .

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
ADC	Analog to Digital Converter
API	Application Programming Interface
ASM	Assembler
AUTOSAR	Automotive Open System Architecture
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

Table 2-3. Reference List

#	Title	Version
1	AUTOSAR 4.0 Rev0003ADC Driver Software Specification Document.	V4.2.0
2	MPC5607B Reference Manual	Rev. 7.2, May 2012
3	MPC5604B Reference Manual	Rev. 8.2, September 2013
4	5606B Reference Manual	Rev. 2, May 2014
5	5602D Reference Manual	Rev. 4.2, Dec 2013

Chapter 3

Building the Driver

This section describes the source files and various compilers, linker options used for building the Autosar ADC driver for Freescale SemiconductorMPC560XB . It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

The ADC driver files are compiled using

- Green Hills MULTI v6.1.4
- Windriver DIAB Rel DIAB_5_9_2_0-FCS_20120725_234511
- Freescale CodeWarrior for MPC5xxx v2.10 - compiler Version 4.3 build 224
- HighTec v4.6.4.0, Compiler gcc v4.6.3

The compiler, linker flags used for building the driver are explained below:

Note

The TS_T2D13M10I1R0 plugin name is composed as follow:

TS_T = Target_Id

D = Derivative_Id

M = SW_Version_Major

I = SW_Version_Minor

R = Revision

(i.e. Target_Id = 2 identifies PowerPC architecture and
Derivative_Id = 13 identifies the MPC560XB)

3.1.1 CW Compiler/Linker/Assembler Options

Table 3-1. Compiler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-lang c	Expects source code to conform to the language specified by the ISO/IEC 9899-1990 ("C90") standard
-opt all	This option is selected all optimization (the same as -opt speed,level=4,intrinsics,noframe)
-common off	Disables moving uninitialized data into a common section
-sdatathreshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small data. (The linker stores small data items in the Small Data address space. The compiler can generate faster code to access this data.)
-sdata2threshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small constant data. (The linker stores small constant data items in the Small Constant Data address space.)
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-use_lmw_stmw on	Enables the use of multiple load and store instructions for function prologues and epilogues
-ppc_asm_to_vle	Converts regular Power Architecture assembler mnemonics to equivalent VLE (Variable Length Encoded) assembler mnemonics in the inline assembler
-Cpp_exceptions off	When on, generates executable code for C++ exceptions. When off, generates smaller, faster executable code
-func_align 4	Specifies alignment of functions in executable code
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format
-w on	Turns on most warning messages
-r	Compiler should expect function prototypes
-w undefmacro	Issues warning messages on the use of undefined macros in #if and #elif conditionals
-char unsigned	Controls the default sign of the char data type: char data items are unsigned
-nosyspath	Performs a search of both the user and system paths, treating #include statements of the form #include xyz the same as the form #include "xyz"
-fp none	No floating point code generation
-common off	Disables moving uninitialized data into a common section
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-2. Assembler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format.
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-3. Linker Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-code_merging all	Removes duplicated functions to reduce object code size
-far_near_addressing	Simplifies address computations to reduce object code size and improve performance
-vle_enhance_merging	Removes duplicated functions that are called by functions that use VLE instructions to reduce object code size
-listdwarf	DWARF debugging information in the linker's map file
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-char unsigned	Controls the default sign of the char data type: char data items are unsigned.

3.1.2 DIAB Compiler/Linker/Assembler Options

Table 3-4. Compiler Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-Xdialect-ansi	Follow the ANSI C standard with some additions
-XO	Enables extra optimizations to produce highly optimized code
-g	Generate symbolic debugger information and disable some optimizations.
-Xsize-opt	Optimize for size rather than speed when there is a choice
-Xsmall-data=0	Set Size Limit for "small data" Variables to zero.
-Xaddr-sconst=0x11	Specify addressing for constant static and global variables with size less than or equal to -Xsmall-const to far-absolute.

Table continues on the next page...

Table 3-4. Compiler Options (continued)

Option	Description
-Xaddr-sdata=0x11	Specify addressing for non-constant static and global variables with size less than or equal to -Xsmall-data in size to far-absolute.
-Xno-common	Disable use of the "COMMON" feature so that the compiler or assembler will allocate each uninitialized public variable in the .bss section for the module defining it, and the linker will require exactly one definition of each public variable
-Xnested-interrupts	Allow nested interrupts
-Xdebug-dwarf2	Generate symbolic debug information in dwarf2 format
-Xdebug-local-all	Force generation of type information for all local variables
-Xdebug-local-cie	Create common information entry per module
-Xdebug-struct-all	Force generation of type information for all typedefs, struct, union and class types
-Xforce-declarations	Generates warnings if a function is used without a previous declaration
-ee1481	Generate an error when the function was used before it has been declared
-Xmacro-undefined-warn	Generates a warning when an undefined macro name occurs in a #if preprocessor directive
-W:as:,-l	Pass the option "-l" (lower case letter L) to the assembler to get an assembler listing file
-Wa,-Xisa-vle	Instruct the assembler to expect and assemble VLE (Variable Length Encoding) instructions rather than BookE instructions.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DDIAB	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the DIAB preprocessor symbol.
-Xforce-prototypes	-Generate warnings if a function is used without a previous prototype declaration.
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries.

Table 3-5. Assembler Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-g	Dump the symbols in the global symbol table in each archive file.
-Xisa-vle	Expect and assemble VLE (Variable Length Encoding) instructions rather than Book E instructions. The default code section is named .text_vle instead of .text, and the default code section fill "character" is set to 0x44444444 instead of 0. The .text_vle code section will have ELF section header flags marking it as VLE code, not Book E code.

Table 3-6. Linker Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-Xelf	Generates ELF object format for output file
-m6	Generates a detailed link map and cross reference table
-lc	Specifies to linker to search for libc.a
-Xlibc-old	Enables usage of legacy (pre-release 5.6) libraries

3.1.3 GHS Compiler/Linker/Assembler Options

Table 3-7. Compiler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-ansi	Specifies ANSI C with extensions. This mode extends the ANSI X3.159-1989 standard with certain useful and compatible constructs.
-noSPE	Disables the use of SPE and vector floating point instructions by the compiler.
-Ospace	Optimize for size
-sda=0	Enables the Small Data Area optimization with a threshold of 0.
-vle	Enables VLE code generation
-dual_debug	Enables the generation of DWARF, COFF, or BSD debugging information in the object file
-G	Generates source level debugging information and allows procedure call from debugger's command line.
--no_exceptions	Disables support for exception handling
-Wundef	Generates warnings for undefined symbols in preprocessor expressions
-Wimplicit-int	Issues a warning if the return type of a function is not declared before it is called
-Wshadow	Issues a warning if the declaration of a local variable shadows the declaration of a variable of the same name declared at the global scope, or at an outer scope
-Wtrigraphs	Issues a warning for any use of trigraphs
--prototype_errors	Generates errors when functions referenced or called have no prototype
--incorrect_pragma_warnings	Valid #pragma directives with wrong syntax are treated as warnings
-noslashcomment	C++ like comments will generate a compilation error
-preprocess_assembly_files	Preprocesses assembly files
-nostartfile	Do not use Start files
--short_enum	Store enumerations in the smallest possible type
--no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options

Table continues on the next page...

Table 3-7. Compiler Options (continued)

Option	Description
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.
-DDISABLE_MCAL_INTERMODULE_ASR_CHECK	-D defines a preprocessor symbol to disable the inter-module version check for AR_RELEASE versions. DISABLE_MCAL_INTERMODULE_ASR_CHECK: By default in the package, drivers are compiled to perform the inter-module version check as per Autosar BSW004. When the inter-module version check needs to be disabled then the DISABLE_MCAL_INTERMODULE_ASR_CHECK global define must be added to the list of compiler options.
-DGHS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the GHS preprocessor symbol.

Table 3-8. Assembler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb

Table 3-9. Linker Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-nostartfiles	Do not use Start files.
-vle	Enables VLE code generation
-linker_warnings	Display linker warnings

3.1.4 HT Compiler/Linker/Assembler Options

Table 3-10. Compiler Options

Option	Description
-Os	Optimize for size. '-Os' enables all '-O2' optimizations that do not typically increase code size. It also performs further optimizations designed to reduce code size.
-msdata=eabi	On System V.4 and embedded PowerPC systems, put small initialized const global and static data in the '.sdata2' section, which is pointed to by register r2. Put small initialized non-const global and static data in the '.sdata' section, which is pointed to by register r13. Put small uninitialized global and static data in the '.sbss' section, which is adjacent to the '.sdata' section.
-G1000	On embedded PowerPC systems, put global and static items less than or equal to num bytes into the small data or bss sections instead of the normal data or bss section.
-fshort-double	Use the same size for double as for float.

Table continues on the next page...

Table 3-10. Compiler Options (continued)

Option	Description
-mno-spe	This option used to disable spe extension (Like –noSPE on ghs compiler).
-mregnames	On System V.4 and embedded PowerPC systems do (do not) emit register names in the assembly language output using symbolic forms
-g3	Request debugging information of Level 3. Includes extra information, such as all the macro definitions present in the program.
-Wall	This enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros.
-mvle	Enable support of VLE
--mcpu=z425n3	Support of e200 Z425n3 core
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.

Table 3-11. Assembler Options

Option	Description
-c	Compile or assemble the source files, but do not link. The linking stage simply is not done. The ultimate output is in the form of an object file for each source file.
-Wa,--gdwarf2	Produce debugging information in DWARF format.
-msdata=eabi	Put small initialized const global and static data in the .sdata2 section, which is pointed to by register r2. Put small initialized non-const global and static data in the .sdata section, which is pointed to by register r13. Put small uninitialized global and static data in the .sbss section, which is adjacent to the .sdata section.
-mregnames	Do emit register names in the assembly language output using symbolic forms.
-mno-spe	This option used to disable spe extension (Like –noSPE on ghs compiler).
-mcpu=z425n3	Support of e200 z425n3 core

Table 3-12. Linker Options

Option	Description
-N	Set the text and data sections to be readable and writable. Also, do not page-align the data segment. If the output format supports Unix style magic numbers, mark the output as OMAGIC.
--no-undefined	Normally when a symbol has an undefined version, the linker will ignore it. This option disallows symbols with undefined version and a fatal error will be issued instead.
--warn-common	Normally ppc-vle-ld will give a warning if it finds an incompatible library during a library search. This option silences the warning.

Table continues on the next page...

Table 3-12. Linker Options (continued)

Option	Description
--error-unresolved-symbols	This restores the linker's default behaviour of generating errors when it is reporting unresolved symbols.
-nocrt0	library crt0 from compiler library not used, it uses custom crt0 file
-nostartfiles	no start file from compiler library is used
-Wl	If the linker is being invoked indirectly, via a compiler driver (e.g. 'gcc') then all the linker command line options should be prefixed by '-Wl,' Used in combination with -no-warn-flags, --no-undefined, --warn-common and --error-unresolved-symbols
--no-warn-flags	Used in combination with -Wl disable linker warnings. See previous
-T	The -T option specifies the name of your own script ie sprig.link. The -c is a synonym of the -T option

3.2 Files required for Compilation

This section describes the include files required to compile, assemble (if assembler code) and link the ADC driver for MPC560XB microcontrollers.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_RELEASE_MAJOR_VERSION and AR_RELEASE_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

ADC Files

- ..\Adc _ TS_T2D13M10I1R0 \src\Adc.c
- ..\Adc _ TS_T2D13M10I1R0 \src\Adc_NonASR.c
- ..\Adc _ TS_T2D13M10I1R0 \src\Adc_ADCCDig_LLD.c
- ..\Adc _ TS_T2D13M10I1R0 \src\Adc_Irq.c
- ..\Adc _ TS_T2D13M10I1R0 \src\Adc_LLD.c
- ..\Adc _ TS_T2D13M10I1R0 \src\Adc_Dma_LLD.c
- ..\Adc _ TS_T2D13M10I1R0 \src\Adc_CTU_LLD.c
- ..\Adc _ TS_T2D13M10I1R0 \src\Reg_eSys_ADCCDig.c
- ..\Adc _ TS_T2D13M10I1R0 \include\Adc.h
- ..\Adc _ TS_T2D13M10I1R0 \include\Adc_NonASR.h
- ..\Adc _ TS_T2D13M10I1R0 \include\Adc_LLD.h
- ..\Adc _ TS_T2D13M10I1R0 \include\Adc_ADCCDig_LLD.h
- ..\Adc _ TS_T2D13M10I1R0 \include\Adc_ADCCDig_LLD_CfgEx.h
- ..\Adc _ TS_T2D13M10I1R0 \include\Adc_CTU_LLD.h
- ..\Adc _ TS_T2D13M10I1R0 \include\Dma_LLD.h
- ..\Adc _ TS_T2D13M10I1R0 \include\Reg_eSys_ADCCDig.h

- ..\Adc_TS_T2D13M10I1R0\include\Reg_eSys_CTU.h
- ..\Adc_TS_T2D13M10I1R0\include\Reg_eSys_DMA.h

ADC Generated Files

- ADC_Cfg.c (For PC Variant) - This file should be generated by the user using a configuration tool for compilation.
- ADC_PBcfg.c (For PB Variant) - This file should be generated by the user using a configuration tool for compilation.
- ADC_Cfg.h - This file should be generated by the user using a configuration tool for compilation.
- ctu_ipversion.h (For PC Variant) - This file should be generated by the user using a configuration tool for compilation.

Files from Base folder

- ..\Base_TS_T2D13M10I1R0\include\Can_GeneralTypes.h
- ..\Base_TS_T2D13M10I1R0\include\Cer.h
- ..\Base_TS_T2D13M10I1R0\include\Compiler.h
- ..\Base_TS_T2D13M10I1R0\include\Compiler_Cfg.h
- ..\Base_TS_T2D13M10I1R0\include\ComStack_Cfg.h
- ..\Base_TS_T2D13M10I1R0\include\ComStack_Types.h
- ..\Base_TS_T2D13M10I1R0\include\Lin_GeneralTypes.h
- ..\Base_TS_T2D13M10I1R0\include\Mcal.h
- ..\Base_TS_T2D13M10I1R0\include\MemMap.h
- ..\Base_TS_T2D13M10I1R0\include\Platform_Types.h
- ..\Base_TS_T2D13M10I1R0\include\Reg_eSys.h
- ..\Base_TS_T2D13M10I1R0\include\Reg_Macros.h
- ..\Base_TS_T2D13M10I1R0\include\Soc_Ips.h
- ..\Base_TS_T2D13M10I1R0\include\Std_Types.h

Files from Dem folder:

- ..\Dem_TS_T2D13M10I1R0\include\Dem.h
- ..\Dem_TS_T2D13M10I1R0\include\Dem_IntErrId.h
- ..\Dem_TS_T2D13M10I1R0\include\Dem_Types.h

Files from Det folder:

- ..\Det_TS_T2D13M10I1R0\include\Det.h

Files from Rte folder:

- ..\Rte_TS_T2D13M10I1R0\src\SchM_Adc.c
- ..\Rte_TS_T2D13M10I1R0\include\SchM_Adc.h

Note:

<plugin_name>: TS_T<2>D<13>M<SW_Version_Major>I<SW_Version_Minor>R0

(i.e. Target_Id = 2 identifies PowerPC architecture and Derivative_Id = 11 identifies the MPC560XB)

3.3 Setting up the Plug-ins

The ADC driver was designed to be configured by using the EB Tresos Studio (version Elektrobit Tresos Release 14.2.1 (b140128-1223) or later.)

Location of various files inside the Adc module folder:

- VSMD (Vendor Specific Module Definition) file in EB tresos Studio XDM format:
 - ..\Adc_TS_T2D13M10I1R0\config\Adc.xdm
 - ..\Mcu_TS_T2D13M10I1R0\config\Mcu.xdm
 - ..\Dem_TS_T2D13M10I1R0\config\Dem.xdm
 - ..\Base_TS_T2D13M10I1R0\config\Base.xdm
 - ..\Resource_TS_T2D13M10I1R0\config\Resource.xdm
- VSMD (Vendor Specific Module Definition) file(s) in AUTOSAR compliant EPD format:
 - ..\Adc_TS_T2D13M10I1R0\autosar\Adc.epd
- Code Generation Templates for Pre-Compile time configuration parameters:
 - ..\Adc_TS_T2D13M10I1R0\generate_PC\include\Adc_Cfg.h
 - ..\Adc_TS_T2D13M10I1R0\generate_PC\include\Adc_CTU_ipversion.h
 - ..\Adc_TS_T2D13M10I1R0\generate_PC\src\Adc_Cfg.c
 - ..\Adc_TS_T2D13M10I1R0\generate_PC\Adc_VersionCheck_Inc.m
 - ..\Adc_TS_T2D13M10I1R0\generate_PC\Adc_VersionCheck_Src.m
 - ..\Adc_TS_T2D13M10I1R0\generate_PC\Adc_RegOperations.m
- Code Generation Templates for Post-Build time configuration parameters:
 - ..\Adc_TS_T2D13M10I1R0\generate_PB\src\Adc_PBcfg.c
 - ..\Adc_TS_T2D13M10I1R0\generate_PB\Adc_RegOperations.m
 - ..\Adc_TS_T2D13M10I1R0\generate_PB\Adc_RegOperations_PB.m
 - ..\Adc_TS_T2D13M10I1R0\generate_PB\Adc_VersionCheck_Src_PB.m

Steps to generate the configuration:

1. Copy the module folders Adc_TS_T2D13M10I1R0, Base_TS_T2D13M10I1R0, Resource_TS_T2D13M10I1R0, Dem_TS_T2D13M10I1R0, Det_TS_T2D13M10I1R0, Mcu_TS_T2D13M10I1R0, Rte_TS_T2D13M10I1R0 into the Tresos plugins folder.
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the EB tresos Studio GUI to modify ECU configuration parameters values.

4. Generate the configuration files.

Dependencies

- **Dem** is required for signaling the production error detection (hardware failure, etc).
- **Det** is required for signaling the development error detection (parameters out of range, null pointers, etc).
- **Mcu** is required for the reference clock and to set DMA channels used by the ADC module.
- **Resource** is required to select processor derivative. Current ADC driver has support for the following derivatives, everyone having attached a Resource file:
 mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64
 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100
 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100
 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144
 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64
 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208
 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100
 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100
 mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176
 mpc5607bxmg_mapbga208.
- **Rte** is required in order to support the critical sections.

Chapter 4

Function calls to module

4.1 Function Calls during Start-up

ADC shall be initialized during STARTUP phase of EcuM initialization. The API to be called for this is `Adc_Init()`. The MCU module and PORT module should be initialized before the ADC is initialized.

Note:

Before to start any ADC conversion, according to the AUTOSAR requirement ADC421, it's mandatory call the function `Adc_SetupResultBuffer`.

4.2 Function Calls during Shutdown

None.

4.3 Function Calls during Wake-up

None.

Chapter 5

Module requirements

5.1 Exclusive areas to be defined in BSW scheduler

In the current implementation, ADC is using the services of Schedule Manager (SchM) for entering and exiting the critical regions. SchM implementation is done by the integrators of the MCAL using OS or non- OS services. For testing the ADC, stubs are used for SchM. All ADC notification functions are called outside any critical region. Global variables updates are performed by ISRs before calling the user notification functions. So the ADC internal state is consistent at the moment of the notification call. The ISR critical regions must not block the other critical regions to avoid deadlocks. This is ensured by exiting the ISR critical region before calling the user notification functions. The following critical regions are used in the ADC driver:

5.1.1 ADC_EXCLUSIVE_AREA_00 Used in function `Adc_EnableGroupNotification`, protects the write to `ADC_GroupStatus[Group].Notification` field. This field is used by other functions.

5.1.2 ADC_EXCLUSIVE_AREA_01 Used in function `Adc_DisableGroupNotification`, protects the write to `ADC_GroupStatus[Group].Notification` field. This field is used by other functions.

5.1.3 ADC_EXCLUSIVE_AREA_02 Used in function `Adc_GetGroupStatus`, protects the read of `ADC_GroupStatus[Group].Conversion` field. This field is used by other functions.

5.1.4 ADC_EXCLUSIVE_AREA_03 Used in function `Adc_DeInit`, protects the read of `ADC_GroupStatus[Group].Conversion` field. This field is used by other functions. No interruptions must occur during this DeInit loop for status check. This exclusive area is executed only if Development Error Detection is enabled.

5.1.5 ADC_EXCLUSIVE_AREA_04 Used in function `Adc_EnableHardwareTrigger`, protects the read and usage of the fields:

- ADC_UnitStatus[unit].QueueIndex
- ADC_UnitStatus[unit].HwQueueIndex

This fields are used by other functions. No interruptions must occur during the read of these variables. This exclusive area is executed only if Development Error Detection is enabled.

5.1.6 ADC_EXCLUSIVE_AREA_05 Used in function Adc_StartGroupConversion, protects the read and usage of the fields:

- ADC_UnitStatus[unit].QueueIndex
- ADC_UnitStatus[unit].HwQueueIndex
- ADC_UnitStatus[unit].Queue[x]
- ADC_GroupStatus[Group].Conversion

This fields are used by other functions.. No interruptions must occur during the read of these variables. This exclusive area is executed only if Development Error Detection is enabled.

5.1.7 ADC_EXCLUSIVE_AREA_06 Used in function ADCDig_LLD_DeInit, protects the read and write of all ADC and CTU registers. This LLD function is called from Adc_DeInit ASR API. No interruptions must occur while these registers are reset.

5.1.8 ADC_EXCLUSIVE_AREA_07 Used in function ADCDig_LLD_StartGroupConversion. This LLD function is called from Adc_StartGroupConversion ASR API. Protects the following:

- Read and write of ADC_UnitStatus[unit].Queue – insert into the queue
- Read and write of ADC_UnitStatus[unit].QueueIndex
- Read and write of ADC registers
- Read and write of DMA regs(disable DMA interrupt)
- Write of ADC_UnitStatus[unit].InjConvOngoin
- Write of ADC_GroupStatus[Group].Conversion

No interruptions must occur while region is running because other ADC functions are also working with the queue and DMA registers.

5.1.9 ADC_EXCLUSIVE_AREA_08 Used in function ADCDig_LLD_StopGroupConversion. This LLD function is called from Adc_StopGroupConversion ASR API. Protects the following:

- Write of ADC_GroupStatus[Group].Conversion
- Write of ADC_UnitStatus[unit].Status
- Write of ADC_GroupStatus[Group].ResultIndex
- Write of ADC_GroupStatus[Group].Notification
- Read and write of ADC_UnitStatus[unit].QueueIndex

- Read and write of ADC_UnitStatus[unit].Queue – remove from queue
- Read and write of ADC registers
- Read and write of DMA registers(disable DMA interrupt)

No interruptions must occur while region is running because other ADC functions are also working with the queue and DMA registers.

5.1.10 ADC_EXCLUSIVE_AREA_09 Used in function ADCDig_LLD_ReadGroup. This LLD function is called from Adc_ReadGroup ASR API. Protects the following:

- Read and write of ADC_GroupStatus[Group].Conversion
- Read and write of ADC_GroupStatus[Group].ResultIndex
- Read of Adc_Cfg_Ptr->Groups[Group].ResultsBufferPtr[Group]

No interruptions must occur while region is running because other ADC functions are also setting the conversion and result index.

5.1.11 ADC_EXCLUSIVE_AREA_10 Used in function ADCDig_LLD_EnableHardwareTrigger. This LLD function is called from Adc_EnableHardwareTrigger ASR API. Protects the following:

- Write to ADC_UnitStatus[unit].HwQueue
- Write to ADC_UnitStatus[unit].HwQueueIndex
- Write to ADC_GroupStatus[Group].Conversion
- Write to ADC_GroupStatus[Group].HwTriggering
- Write to ADC_GroupStatus[Group].ResultIndex
- Write to ADC registers
- Write to CTU registers

No interruptions must occur while region is running because other ADC functions are also setting the fields and ADC/CTU registers.

5.1.12 ADC_EXCLUSIVE_AREA_11 Used in function ADCDig_LLD_DisableHardwareTrigger. This LLD function is called from Adc_DisableHardwareTrigger ASR API. Protects the following:

- Write to ADC_GroupStatus[Group].Notification
- Write to ADC_GroupStatus[Group].HwTriggering
- Write to CTU registers
- Write to ADC registers
- Write to ADC_GroupStatus[Group].Conversion
- Write to ADC_UnitStatus[unit].HwQueue[0]
- Write to ADC_UnitStatus[unit].HwQueueIndex

No interruptions must occur while region is running because other ADC functions are also setting the fields and ADC/CTU registers.

5.1.13 ADC_EXCLUSIVE_AREA_12 Used in function ADCDig_LLD_GetStreamLastPointer. This LLD function is called from Adc_GetStreamLastPointer ASR API. Protects the following:

- Read and write to ADC_GroupStatus[Group].Conversion
- Read and write to ADC_GroupStatus[Group].ResultIndex

No interruptions must occur while region is running because other ADC functions are also setting the fields.

5.1.14 ADC_EXCLUSIVE_AREA_13 Used in ISR processing function Adc_DMAEndGroupConversion, and covers also the call to Adc_DMAEndNormalConv. It protects the following:

- Write of DMA registers
- Read of ADC_UnitStatus[unit].Queue
- Calls Adc_DMAEndNormalConv:
 - Read and write to ADC_GroupStatus[Group].Conversion
 - Read and write to ADC_GroupStatus[Group].ResultIndex
 - Read and write to ADC registers
 - Read and write to ADC_UnitStatus[unit].QueueIndex
 - Write ADC_UnitStatus[unit].Status

This region must protect against interruptions by other functions that can set this fields.

5.1.15 ADC_EXCLUSIVE_AREA_14 Used in ISR processing function Adc_EndNormalConv, protects the following:

- Read and write to ADC_UnitStatus[unit].Queue
- Read and write to ADC_GroupStatus[Group].ResultIndex
- Write to Adc_Cfg_Ptr->Groups[Group].ResultsBufferPtr
- Read and write to ADC_GroupStatus[Group].Conversion
- Write to ADC registers
- Read and write to ADC_UnitStatus[unit].Status
- Read and write to ADC_UnitStatus[unit].QueueIndex

This region must protect against interruptions by other functions that can set this fields.

5.1.16 ADC_EXCLUSIVE_AREA_15 Used in ISR processing function Adc_EndInjectedConv, protects the following:

- Read of ADC_UnitStatus[unit].HwQueue
- Write of Adc_Cfg_Ptr->Groups[Group].ResultsBufferPtr
- Read and write ADC_GroupStatus[Group].ResultIndex
- Read and write ADC_GroupStatus[Group].Conversion
- Write to ADC registers

- Write of ADC_UnitStatus[unit].HwQueueIndex
- Write of ADC_UnitStatus[unit].InjConvOngoin

This region must protect against interruptions by other functions that can set this fields.

5.1.17 ADC_EXCLUSIVE_AREA_16 Used in ISR processing function

Adc_EndCTUConv, protects the following:

- Read of ADC_UnitStatus[unit].HwQueue
- Write of Adc_Cfg_Ptr->Groups[Group].ResultsBufferPtr
- Write of CTU registers
- Write of ADC registers
- Write of ADC_GroupStatus[Group].ResultIndex
- Write of ADC_GroupStatus[Group].Conversion
- Write of ADC_UnitStatus[unit].Status
- Write of ADC_UnitStatus[unit].HwQueueIndex
- Write of ADC_UnitStatus[unit].InjConvOngoin

This region must protect against interruptions by other functions that that can set this fields.

5.1.18 ADC_EXCLUSIVE_AREA_17 Used in function Adc_GetStreamLastPointer, protects the read of ADC_GroupStatus[Group].Conversion field. This field is updated by other ADC functions.

5.1.19 ADC_EXCLUSIVE_AREA_18 Used in function Adc_ReadGroup, protects the read of ADC_GroupStatus[Group].Conversion field. This field is updated by other ADC functions.

5.1.20 ADC_EXCLUSIVE_AREA_19 Used in function Adc_StopGroupConversion, protects the read of ADC_GroupStatus[Group].Conversion field. This field is updated by other ADC functions.

5.1.21 ADC_EXCLUSIVE_AREA_20 Used in function Adc_EnableHwTrigger, protects the following:

- Read of ADC_UnitStatus[unit].HwQueueIndex
- Read of ADC_UnitStatus[unit].QueueIndex

5.1.22 ADC_EXCLUSIVE_AREA_21 Used in function AdcDig_LLD_EnableHwTrigger. This LLD function is called from Adc_EnableHwTrigger ASR API. Protects the following:

- Write to ADC_GroupStatus[Group].Conversion
- Write to ADC_GroupStatus[Group].HwTriggering
- Write to ADC_UnitStatus[unit].HwQueue

- Write to ADC_UnitStatus[unit].HwQueueIndex
- Write to to ADC registers
- Write to to CTU registers

5.1.23 ADC_EXCLUSIVE_AREA_22 Used in function AdcDig_LLD_DisableHwTrigger. This LLD function is called from Adc_EnableHwTrigger ASR API. Protects the following:

- Write to ADC_GroupStatus[Group].Conversion
- Write to ADC_GroupStatus[Group].HwTriggering
- Write to ADC_UnitStatus[unit].HwQueue
- Write to ADC_UnitStatus[unit].HwQueueIndex
- Write to to ADC registers
- Write to to CTU registers

5.1.24 ADC_EXCLUSIVE_AREA_23 Used in function ADCDig_LLD_HwResultReadGroup. This LLD function is called from Adc_HwResultReadGroup ASR API. Protects the following:

- Read of ADC registers

5.1.25 ADC_EXCLUSIVE_AREA_24 Used in function Adc_EndSwInjectedConv, protects the following:

- Read of ADC_UnitStatus[unit].SwInjQueue
- Write of Adc_Cfg_Ptr->Groups[Group].ResultsBufferPtr
- Read and write ADC_GroupStatus[Group].ResultIndex
- Read and write ADC_GroupStatus[Group].Conversion
- Write to ADC registers
- Write of ADC_UnitStatus[unit].SwInjQueueIndex
- Write of ADC_UnitStatus[unit].InjConvOngoin

5.1.26 ADC_EXCLUSIVE_AREA_25 Used in both the functions Non ASR Adc_EnableChannel and Adc_DisableChannel to protect the write to the Adc_NCMRx_Mask structure. The fields of this structure are updated by other ADC functions.

5.1.27 ADC_EXCLUSIVE_AREA_26 Used in the function Non ASR function Adc_ConfigureThreshold. This region protects against interruptions by other functions from writing to the ADC watchdog threshold registers.

5.2 Peripheral Hardware Requirements

The device provides two precision Analog to Digital Converter HW units: ADC HW Unit 0 at 10-bit and ADC HW Unit 1 at 12-bit. The number of channels are derivative specific, so please consult the derivative manuals.

5.3 ISR to configure within OS – dependencies

The following ISR's are used by the ADC driver:

Table 5-1. ADC ISR

ISR Name	HW INT Vector	Observations
ISR(Adc_EndGroupConvUnit0)	62	for Bolero 512K and Bolero 1.5M
ISR(Adc_DMAEndGroupConvUnit0)	11	DMA channel 0; for Bolero 512K and Bolero 1.5M
ISR(Adc_WatchDogThresholdUnit0)	64	for Bolero 512K and Bolero 1.5M
ISR(Adc_EndGroupConvUnit1)	82	only for Bolero 1.5M
ISR(Adc_DMAEndGroupConvUnit1)	12	DMA channel 1; only for Bolero 1.5M
ISR(Adc_WatchDogThresholdUnit1)	84	only for Bolero 1.5M

5.4 ISR Macro

MCAL drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions:

a. OS is not used - AUTOSAR_OS_NOT_USED is defined:

i. If USE_SW_VECTOR_MODE is defined:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, drivers' interrupt handlers are normal C functions and the prolog/epilog handle the context save and restore.

ii. If USE_SW_VECTOR_MODE is not defined:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, drivers' interrupt handlers must save and restore the execution context.

Custom OS is used - AUTOSAR_OS_NOT_USED is not defined

```
#define ISR(IsrName) void OS_isr_##IsrName()
```

In this case, OS is handling the execution context when an interrupt occurs. Drivers' interrupt handlers are normal C functions.

Other vendor's OS is used - AUTOSAR_OS_NOT_USED is not defined. Please refer to the OS documentation for description of the ISR macro.

5.5 Other AUTOSAR modules - dependencies

- **Mcu** The Microcontroller Unit Driver (MCU Driver) is primarily responsible for initializing and controlling the chips internal clock sources and clock prescalers. The clock frequency may affect the Trigger frequency, Conversion time and Sampling time. In DMA mode, the MCU is used to configure the DMA channel allocated for both ADC HW units.
- **Det** If development error detection for the ADC module is enabled: The ADC module shall raise errors to the Development Error Tracer (DET) whenever a development error is encountered by this module.
- **Dem:** The ADC module shall report production errors to the Diagnostic Event Manager (DEM).
- **Port:** The PORT module shall configure the port pins used by the ADC module. Both analogue input pins and external trigger pins have to be considered.
- **Resource:** Sub-Derivative model is selected from Resource configuration.
- **RTE:** Used to manage the exclusive area inside Adc module.

Chapter 6

Main API Requirements

6.1 Main functions calls within BSW scheduler, at which cycle (10ms, 20ms,...?)

None.

6.2 API Requirements

None.

6.3 Calls to Notification Functions, Callbacks, Callouts

Call-back Notifications:

None

User Notification:

The ADC Driver provides a notification callback per channel that is called whenever the defined time period is over. The notifications can be configured as pointers to user defined functions. If notification is not desired, 'NULL_PTR' shall be configured.

The syntax of this function is as follows:

```
void Adc_Notification_<channel>()
```

An extern declaration of this function is available in Adc_PBcfg.c. The function has to be implemented by the user.

Chapter 7

Memory Allocation

7.1 Sections to be defined in MemMap.h

For Post Build data:

```
#ifndef ADC_START_CONFIG_DATA_UNSPECIFIED
#define ADC_START_CONFIG_DATA_UNSPECIFIED
#define MEMMAP_ERROR
/*Memory Section for Post Build Data to be defined here.
   Example given in the next line*/
#pragma ghs section rodata=".mcal_const_cfg"
#endif
#ifndef ADC_STOP_CONFIG_DATA_UNSPECIFIED
#define ADC_STOP_CONFIG_DATA_UNSPECIFIED
#define MEMMAP_ERROR
/*End of section to be mentioned here. Example given in the
   next line.*/
#pragma ghs section
#endif
```

For Code:

```
#ifndef ADC_START_SEC_CODE
#define ADC_START_SEC_CODE
#define MEMMAP_ERROR
/*Memory Section for Code to be defined here.*/
#pragma ghs section text=".mcal_text"
#endif
#ifndef ADC_STOP_SEC_CODE
#define ADC_STOP_SEC_CODE
#define MEMMAP_ERROR
/*End of section to be mentioned here*/
#endif
```

For Initialized Variables:

```
#ifndef ADC_START_SEC_VAR_INIT_UNSPECIFIED
#define ADC_START_SEC_VAR_INIT_UNSPECIFIED
#define MEMMAP_ERROR
/*Memory Section for Variables to be defined here.*/
#pragma ghs section data=".mcal_data"
```

Linker command file

```
#endif
#ifdef ADC_STOP_SEC_VAR_INIT_UNSPECIFIED
#undef ADC_STOP_SEC_VAR_INIT_UNSPECIFIED
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#endif
```

For Uninitialized Variables:

```
#ifdef ADC_START_SEC_VAR_NO_INIT_UNSPECIFIED
#undef ADC_START_SEC_VAR_NO_INIT_UNSPECIFIED
#undef MEMMAP_ERROR
/*Memory Section for Variables to be defined here.*/
#pragma ghs section bss=".mcal_bss"
#endif
#ifdef ADC_STOP_SEC_VAR_NO_INIT_UNSPECIFIED
#undef ADC_STOP_SEC_VAR_NO_INIT_UNSPECIFIED
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#endif
```

7.2 Linker command file

Memory shall be allocated for every section defined in MemMap.h.

Chapter 8

Configuration parameters considerations

Configuration parameter class for Autosar ADC driver fall into the following variants as defined in table.

8.1 Configuration parameters considerations

Configuration parameter class for Autosar ADC driver fall into the following variants as defined below:

Table 8-1. Configuration Parameters

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
ADC General			
	AdcDevErrorDetect	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcEnableLimitCheck	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcVersionInfoApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcSetModeApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcDelInitApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcEnableStartStopGroupApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcHwTriggerApi	Pre Compile parameter for all Variants of Configuration	Pre Compile

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
	AdcGrpNotifCapability	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcPriorityImplementation	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcReadGroupApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcResultAlignment	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcConvTimeOnce	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcPreSamplingOnce	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcSetOnceRegisters	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcTimeout	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcEnableQueueing	Pre Compile parameter for all Variants of Configuration	Pre Compile
AdcConfigSet - AdcGeneric			
	AdcPriorityQueueMaxDepth	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcTransferType	Pre Compile parameter for all Variants of Configuration	Pre Compile
	Adc0MaxGroupChannels	Pre Compile parameter for all Variants of Configuration	Pre Compile
	Adc1MaxGroupChannels	Pre Compile parameter for all Variants of Configuration	Pre Compile
ADCHwUnit- General			
	AdcClockSource	VariantPC or VariantPB	Post Build
	AdcHwUnitId	VariantPC or VariantPB	Post Build
	AdcPrescale	VariantPC or VariantPB	Post Build
	AdcPowerDownDelay	VariantPC or VariantPB	Post Build
	AdcMuxDelay	VariantPC or VariantPB	Post Build

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
	AdcAutoClockOff	VariantPC or VariantPB	Post Build
	AdcPresamplingEnableSignal	VariantPC or VariantPB	Post Build
	AdcBypassSampling	VariantPC or VariantPB	Post Build
ADCChannel- General			
	AdcChannelId	VariantPC or VariantPB	Post Build
	AdcChannelConvTime	VariantPC or VariantPB	Pre Compile
	AdcChannelHighLimit	VariantPC or VariantPB	Pre Compile
	AdcChannelRefVoltsrcLow	VariantPC or VariantPB	Post Build
	AdcChannelRefVoltsrcHigh	VariantPC or VariantPB	Post Build
	AdcHwChannel	VariantPC or VariantPB	Post Build
	AdcChannelResolution	VariantPC or VariantPB	Post Build
	AdcChannelSampTime	VariantPC or VariantPB	Post Build
	AdcEnablePresampling	VariantPC or VariantPB	Post Build
	AdcEnableThresholds	VariantPC or VariantPB	Post Build
	AdcChannelLimitCheck	VariantPC or VariantPB	Post Build
	AdcChannelLowLimit	VariantPC or VariantPB	Post Build
	AdcChannelRangeSelect	VariantPC or VariantPB	Post Build
ADCChannel- AdcThresholdControl			
	AdcThresholdControlRegister	VariantPC or VariantPB	Post Build
	AdcHighThreshold	VariantPC or VariantPB	Post Build
	AdcLowThreshold	VariantPC or VariantPB	Post Build
	AdcWdogNotification	VariantPC or VariantPB	Post Build
ADCGroup- General			
	AdcGroupAccessMode	VariantPC or VariantPB	Post Build
	AdcGroupConversionMode	VariantPC or VariantPB	Post Build
	AdcGroupConversionType	VariantPC or VariantPB	Post Build
	AdcGroupId	VariantPC or VariantPB	Post Build
	AdcGroupPriority	VariantPC or VariantPB	Post Build
	AdcGroupReplacement	VariantPC or VariantPB	Post Build
	AdcGroupTriggSrc	VariantPC or VariantPB	Post Build
	AdcHwTrigSignal	VariantPC or VariantPB	Post Build
	AdcHwTrigTimer	VariantPC or VariantPB	Post Build
	AdcNotification	VariantPC or VariantPB	Post Build
	AdcStreamingBufferMode	VariantPC or VariantPB	Pre Compile
	AdcResultBufferPointer	VariantPC or VariantPB	Post Build
	AdcStreamingNumSamples	VariantPC or VariantPB	Post Build
	AdcEnableChDisableChGroup	VariantPC or VariantPB	Post Build

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
	AdcWithoutInterrupts	VariantPC or VariantPB	Post Build
	AdcMultipleHardwareTriggerGroup	VariantPC or VariantPB	Post Build
AdcHwTrig			
	AdcHwTrigSrc	VariantPC or VariantPB	Post Build
AdcGroupConversionConfiguration			
	AdcLatchingDuration	VariantPC or VariantPB	Post Build
	AdcComparisonDuration	VariantPC or VariantPB	Post Build
	AdcSamplingDuration	VariantPC or VariantPB	Post Build
ADCGroup- AdcGroupDefinition			
	AdcGroupDefinition	VariantPC or VariantPB	Post Build
AdcPublishedInformation			
	AdcChannelValueSigned	VariantPC or VariantPB	Post Build
	AdcGroupFirstChannelFixed	VariantPC or VariantPB	Post Build
	AdcMaxChannelResolution	VariantPC or VariantPB	Post Build
CommonPublishedInformation			
	VendorId	AdcGroupFirstChannelFixed	Post Build
	ModuleId	VariantPC or VariantPB	Post Build
	ArMajorVersion	VariantPC or VariantPB	Post Build
	ArMinorVersion	VariantPC or VariantPB	Post Build
	ArPatchVersion	VariantPC or VariantPB	Post Build
	SwMajorVersion	VariantPC or VariantPB	Post Build
	SwMinorVersion	VariantPC or VariantPB	Post Build
	SwPatchVersion	VariantPC or VariantPB	Post Build
	VendorApilnFix	VariantPC or VariantPB	Post Build
NonAutosar			
	AdcEnableCtuTrigNonAutosarApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcSetModeApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcEnableDualClockMode	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcEnableGroupDependentChannelNames	Pre Compile parameter for all Variants of Configuration	Pre Compile

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
	AdcBypassConsistencyLoop	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcEnableChDisableChApi	Pre Compile parameter for all Variants of Configuration	Pre Compile
	AdcGetInjectedConvStatusApi	Pre Compile parameter for all Variants of Configuration	Pre Compile

Chapter 9

Integration Steps

This section gives a brief overview of the steps needed for integrating Analogue to Digital Converter Driver :

- Generate the required ADC configurations. For more details refer to section [Files required for Compilation](#)
- Allocate proper memory sections in MemMap.h and linker command file. For more details refer to section [Sections to be defined in MemMap.h](#)
- Make sure all include files for compilation are as per section [ISR Reference](#)
- Map the ISRs to their vector locations. For more details refer to section [ISR to configure within OS – dependencies](#)
- Compile & build the ADC with all the dependent modules. For more details refer to section [Building the Driver & Other AUTOSAR modules - dependencies](#)



Chapter 10

ISR Reference

ISR functions exported by the ADC driver.

10.1 Software specification

The following sections contains driver software specifications.

10.1.1 Define Reference

Constants supported by the ADC driver.

10.1.2 Macros Reference

Macros supported by the ADC driver.

10.1.2.1 Macro ADC_CTU_START_VALUE

macros to simplify access to structures.

Prototype: `ADC_CTU_START_VALUE (num)`

10.1.2.2 Macro ADC_GROUP_PRIORITY

Adc group priority.

Prototype: `ADC_GROUP_PRIORITY (Priority)`

10.1.2.3 Macro ADC_GroupInQueue

The group at position p in queue of given hardware unit.

Prototype: `ADC_GroupInQueue(unit, p)`

10.1.2.4 Macro ADC_GroupsInHwQueue

Number of hardware groups in queue for hardware unit.

Prototype: `ADC_GroupsInHwQueue(unit)`

10.1.2.5 Macro ADC_GroupTriggerSource

Adc hardware trigger source configured for the group.

Prototype: `ADC_GroupTriggerSource(group, pos)`

10.1.2.6 Macro ADC_GroupUnit

Configured hardware unit of the group.

Prototype: `ADC_GroupUnit(group)`

10.1.2.7 Macro ADC_STREAMING_NUM_SAMPLES

macros to simplify access to structures.

Details:

Streaming number of samples

Prototype: `ADC_STREAMING_NUM_SAMPLES(num)`

10.1.2.8 Macro CTU_LLD_ClearTRGI

This macro clears the TRGIE bit in the CTU_CSR register.

Details:

Clears the TRGIE field in the CTU_CSR register in order to ack the interrupt.

Implements:

Prototype: `CTU_LLD_ClearTRGI()`

10.1.2.9 Macro CTU_LLD_Init_FALSE

Prototype: `CTU_LLD_Init_FALSE()`

10.1.2.10 Macro CTU_LLD_Init_TRUE

This macro initializes the CTU hardware module.

Details:

Sets TRGIEN bit, in conformance with the parameter, from the CTU_CSR register.

Implements:

Prototype: `CTU_LLD_Init_TRUE()`

10.1.2.11 Macro CTU_LLD_ReadEvtCfgRegs

This macro read the CTU hardware module CTU_EVTCTFGRx registers.

Details:

Read the CTU_EVTCTFGRx register and mask with the Trigger Mask bit.

Implements:

Prototype: `CTU_LLD_ReadEvtCfgRegs(TriggSrc)`

10.1.2.12 Macro CTU_LLD_ResetEvtCfgRegs

This macro initializes the CTU hardware module CTU_EVTCTFGRx registers.

Details:

Resets all CTU_EVTCFGGRx registers (write 0 for reset value).

Implements:

Prototype: CTU_LLD_ResetEvtCfgRegs()

10.1.2.13 Macro CTU_LLD_ResetPrescalerBits

This macro initializes the CTU hardware module.

Details:

Resets PRESC_CONF bits from the CTU_CSR register (write 0 for reset value).

Implements:

Prototype: CTU_LLD_ResetPrescalerBits()

10.1.2.14 Macro CTU_LLD_ResetSvrRegs

This macro initializes the CTU hardware module CTU_SVRx registers.

Details:

Resets all CTU_SVRx registers (write 0 for reset value).

Implements:

Prototype: CTU_LLD_ResetSvrRegs()

10.1.2.15 Macro CTU_LLD_SetCounterStartValue

This macro initializes the CTU hardware module CTU_SVRx registers.

Details:

Sets the CTU_SVRx register, in conformance with the parameter.

Implements:

Prototype: CTU_LLD_SetCounterStartValue(Index, CounterStartValue)

10.1.2.16 Macro CTU_LLD_SetEvtCfgReg

Configures the CTU hardware module CTU_EVTCTFGRx register with the desired values.

Details:

Updates all fields in the CTU_EVTCTFGRx register (the register is specified by the TriggerSource parameter) AdcHwUnitSelector is uint8 because on some future platforms can be more than 2 ADC hw units.

- N.B.: Bolero 256k-128k has 1 ADC hw unit.

Implements:

Prototype:

CTU_LLD_SetEvtCfgReg(TriggerSource, TriggerMask, ClearFlag, CountGroup, DelayIndex, AdcChannelID)

Table 10-1. CTU_LLD_SetEvtCfgReg Arguments

Name	Direction	Description
TriggerSource	input	Specifies the trigger source index (ch0-eMIOS0, ch32-PIT_3).
TriggerMask	input	The value of the Trigger Mask field: 0: Trigger masked 1: Trigger enabled.
ClearFlag	input	The value of the CLR_FLAG field: To provide flag_ack through software 1: Flag_ack is forced to '1' for the particular event 0: Flag_ack is dependent on flag servicing This bit implementation is generic based and implemented only for inputs mapped to PIT event flags.
CountGroup	input	The value of the COUNT_GROUP field: 00: Counter 0 is associated with the particular event 01: Counter 1 is associated with the particular event 10: Counter 2 is associated with the particular event

Table continues on the next page...

**Table 10-1. CTU_LLD_SetEvtCfgReg Arguments
(continued)**

Name	Direction	Description
		11: Counter 3 is associated with the particular event.
DelayIndex	input	The value of the DELAY_INDEX field: 000: No delay is provided 001: Counter is loaded with value stored in CTU_SVR1 010: Counter is loaded with value stored in CTU_SVR2 011: Counter is loaded with value stored in CTU_SVR3 100: Counter is loaded with value stored in CTU_SVR4 101: Counter is loaded with value stored in CTU_SVR5 110: Counter is loaded with value stored in CTU_SVR6 111: Counter is loaded with value stored in CTU_SVR7.
AdcChannelID	input	The ID of the ADC channel (in order to implement the ADC channel mapping over the CTU channels) CHANNEL_VALUE[x:0] (the precision is platform dependent) These bits provide the ADC channel number to be converted. Valid values are 0b0 to 0b1011111 (decimal 95).

10.1.2.17 Macro CTU_LLD_SetPrescalerValue

This macro initializes the CTU hardware module.

Details:

Sets PRESC_CONF bits, in conformance with the parameter, from the CTU_CSR register.

Implements:

Prototype: `CTU_LLD_SetPrescalerValue(PrescConfig)`

10.1.3 Enum Reference

Enumeration of all constants supported by the ADC driver.

10.1.4 Function Reference

Functions supported by the ADC driver.

10.1.4.1 Function `Adc_DMAEndGroupConvUnit0`

This function implements the ISR for the conversion done of the ADC Hardware unit 0.



Figure 10-1. Function `Adc_DMAEndGroupConvUnit0` References.

Details:

The function implements the ISR for the ADC Hardware unit 0.

Return: void.

Implements: DADC04018

Note

This ISR is only defined if the `ADC_DMA_SUPPORTED` is enabled.

Prototype: `void Adc_DMAEndGroupConvUnit0(void);`

10.1.4.2 Function `Adc_DMAEndGroupConvUnit1`

This function implements the ISR for the conversion done of the ADC Hardware unit 1.



Figure 10-2. Function Adc_DMAEndGroupConvUnit1 References.

Details:

The function implements the ISR for the ADC Hardware unit 1.

Return: void.

Implements: DADC04021

Note

This ISR is only defined if the ADC_DMA_SUPPORTED is enabled.

Prototype: void Adc_DMAEndGroupConvUnit1(void);

10.1.4.3 Function Adc_EndGroupConvUnit0

This function implements the ISR for the conversion done of the ADC Hardware unit 0.



Figure 10-3. Function Adc_EndGroupConvUnit0 References.

Details:

The function implements the ISR for the ADC Hardware unit 0.

Return: void.

Implements: DADC04016

Prototype: void Adc_EndGroupConvUnit0(void);

10.1.4.4 Function `Adc_EndGroupConvUnit1`

This function implements the ISR for the conversion done of the ADC Hardware unit 1.



Figure 10-4. Function `Adc_EndGroupConvUnit1` References.

Details:

The function implements the ISR for the ADC Hardware unit 1.

Return: void.

Implements: DADC04019

Prototype: void `Adc_EndGroupConvUnit1(void);`

10.1.4.5 Function `Adc_WatchDogThresholdUnit0`

This function implements the ISR for the conversion done of the ADC Hardware unit 0.



Figure 10-5. Function `Adc_WatchDogThresholdUnit0` References.

Details:

The function implements the ISR for the ADC Hardware unit 0.

Return: void.

Implements: DADC04017

Note

This ISR is only defined if the `ADC_WDG_SUPPORTED` is enabled.

Prototype: void `Adc_WatchDogThresholdUnit0(void);`

10.1.4.6 Function `Adc_WatchDogThresholdUnit1`

This function implements the ISR for the conversion done of the ADC Hardware unit 1.



Figure 10-6. Function `Adc_WatchDogThresholdUnit1` References.

Details:

The function implements the ISR for the ADC Hardware unit 1.

Return: void.

Implements: DADC04020

Note

This ISR is only defined if the `ADC_WDG_SUPPORTED` is enabled.

Prototype: `void Adc_WatchDogThresholdUnit1(void);`

10.1.5 Structs Reference

Data structures supported by the ADC driver.

10.1.6 Types Reference

Types structures supported by the ADC driver.

10.1.7 Variables Reference

Variables supported by the ADC driver.

How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number IM13ADCASR4.0 Rev0003R1.0.1
Revision 1.2